

2011. Final Value of Variable After Performing Operations

Easy | Topics | Companies | Hint

There is a programming language with only **four** operations and **one** variable `X`:

- `++X` and `X++` **increments** the value of the variable `X` by `1`.
- `--X` and `X--` **decrements** the value of the variable `X` by `1`.

Initially, the value of `X` is `0`.

Given an array of strings `operations` containing a list of operations, return *the final value of `X` after performing all the operations*.

Example 1:

Input: `operations = ["--X","X++","X++]`

Output: `1`

Explanation: The operations are performed as follows:
Initially, `X = 0`.

`--X`: `X` is decremented by `1`, `X = 0 - 1 = -1`.

`X++`: `X` is incremented by `1`, `X = -1 + 1 = 0`.

`X++`: `X` is incremented by `1`, `X = 0 + 1 = 1`.

Example 2:

Input: `operations = ["++X","++X","X++]`

Output: `3`

Explanation: The operations are performed as follows:
Initially, `X = 0`.

`++X`: `X` is incremented by `1`, `X = 0 + 1 = 1`.

`++X`: `X` is incremented by `1`, `X = 1 + 1 = 2`.

`X++`: `X` is incremented by `1`, `X = 2 + 1 = 3`.

Code

Java | Auto

```
1 class Solution {
2     public int finalValueAfterOperations(String[] operations) {
3         int x = 0;
4         for (String op : operations) {
5             if (op.charAt(1) == '+') {
6                 x++;
7             } else {
8                 x--;
9             }
10        }
11        return x;
12    }
13 }
```

Saved

Ln 4, Col 39

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

operations =
["--X","X++","X++]

Output

1

Expected

1

2180. Count Integers With Even Digit Sum

Easy

Topics

Companies

Hint

Given a positive integer `num`, return the number of positive integers **less than or equal to** `num` whose digit sums are **even**.

The **digit sum** of a positive integer is the sum of all its digits.

Example 1:

Input: num = 4

Output: 2

Explanation:

The only integers less than or equal to 4 whose digit sums are even are 2 and 4.

Example 2:

Input: num = 30

Output: 14

Explanation:

The 14 integers less than or equal to 30 whose digit sums are even are 2, 4, 6, 8, 11, 13, 15, 17, 19, 20, 22, 24, 26, and 28.

Constraints:

- `1 <= num <= 1000`

Code

Java

Auto

```
1 class Solution {
2     public int countEven(int num) {
3         int temp = num;
4         int digitSum = 0;
5         while (temp > 0) {
6             digitSum += temp % 10;
7             temp /= 10;
8         }
9         return digitSum % 2 == 0 ? num / 2 : (num - 1) / 2;
10    }
11 }
```

Saved

Ln 4, Col 26

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

num =

4

Output

2

Expected

2

1945. Sum of Digits of String After Convert

Easy | Topics | Companies | Hint

You are given a string `s` consisting of lowercase English letters, and an integer `k`. Your task is to *convert* the string into an integer by a special process, and then *transform* it by summing its digits repeatedly `k` times. More specifically, perform the following steps:

1. **Convert** `s` into an integer by replacing each letter with its position in the alphabet (i.e. replace 'a' with 1, 'b' with 2, ..., 'z' with 26).
2. **Transform** the integer by replacing it with the **sum of its digits**.
3. Repeat the **transform** operation (step 2) `k` **times** in total.

For example, if `s = "zbx"` and `k = 2`, then the resulting integer would be 8 by the following operations:

1. **Convert:** `"zbx"` → `"(26)(2)(1)(24)"` → `"262124"` → 262124
2. **Transform #1:** 262124 → 2 + 6 + 2 + 1 + 2 + 4 → 17
3. **Transform #2:** 17 → 1 + 7 → 8

Return the **resulting integer** after performing the **operations** described above.

Example 1:

Input: `s = "iiii", k = 1`

Output: 36

Explanation:

The operations are as follows:

Code

Java | Auto

```
int position = 0;
while (position > 0) {
    sum += position % 10;
    position /= 10;
}
for (int i = 1; i < k; i++) {
    int currentSum = 0;
    while (sum > 0) {
        currentSum += sum % 10;
        sum /= 10;
    }
    sum = currentSum;
}
```

Saved

Ln 5, Col 40

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s =
"iiii"

k =
1

Output

36

2413. Smallest Even Multiple

Solved

Easy

Topics

Companies

Hint

Given a **positive** integer `n`, return *the smallest positive integer that is a multiple of both `2` and `n`*.

Example 1:

Input: `n = 5`
Output: `10`
Explanation: The smallest multiple of both 5 and 2 is 10.

Example 2:

Input: `n = 6`
Output: `6`
Explanation: The smallest multiple of both 6 and 2 is 6. Note that a number is a multiple of itself.

Constraints:

- `1 <= n <= 150`

Which Company? 2/6

Google

Amazon

Bloomberg

Meta

Microsoft

Search...

Confirm

Accepted 302 865 / 342 6K

Acceptance Rate 88.4%

1.1K

68

☆

?

8 Online

Code

Java

Auto

```
1 class Solution {
2     public int smallestEvenMultiple(int n) {
3         if(n%2==0) return n;
4         return n*2;
5     }
6 }
```

Saved

Ln 4, Col 20

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

n =

5

Output

10

Expected

10

2651. Calculate Delayed Arrival Time

Easy

Topics

Companies

Hint

You are given a positive integer `arrivalTime` denoting the arrival time of a train in hours, and another positive integer `delayedTime` denoting the amount of delay in hours.

Return *the time when the train will arrive at the station*.

Note that the time in this problem is in 24-hours format.

Example 1:

Input: `arrivalTime = 15, delayedTime = 5`

Output: `20`

Explanation: Arrival time of the train was 15:00 hours. It is delayed by 5 hours. Now it will reach at 15+5 = 20 (20:00 hours).

Example 2:

Input: `arrivalTime = 13, delayedTime = 11`

Output: `0`

Explanation: Arrival time of the train was 13:00 hours. It is delayed by 11 hours. Now it will reach at 13+11=24 (Which is denoted by 00:00 in 24 hours format so return 0).

Constraints:

- $1 \leq \text{arrivalTime} < 24$
- $1 \leq \text{delayedTime} \leq 24$

</> Code

Java ▾ Auto



```
1 class Solution {
2     public int findDelayedArrivalTime(int arrivalTime, int delayedTime) {
3         return (arrivalTime + delayedTime) % 24;
4     }
5 }
```

Saved

☒ Testcase | Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2

Input

arrivalTime =
15

delayedTime =
5

Output

20

1365. How Many Numbers Are Smaller Than the Current Number

Easy

Topics

Companies

Hint

Given the array `nums`, for each `nums[i]` find out how many numbers in the array are smaller than it. That is, for each `nums[i]` you have to count the number of valid `j`'s such that `j != i` and `nums[j] < nums[i]`.

Return the answer in an array.

Example 1:

Input: `nums = [8,1,2,2,3]`

Output: `[4,0,1,1,3]`

Explanation:

For `nums[0]=8` there exist four smaller numbers than it (1, 2, 2 and 3).

For `nums[1]=1` does not exist any smaller number than it.

For `nums[2]=2` there exist one smaller number than it (1).

For `nums[3]=2` there exist one smaller number than it (1).

For `nums[4]=3` there exist three smaller numbers than it (1, 2 and 2).

Example 2:

Input: `nums = [6,5,4,8]`

Output: `[2,1,0,3]`

Example 3:

Input: `nums = [7,7,7,7]`

Output: `[0,0,0,0]`

Code

Java | Auto

```
1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3         int[] count = new int[102];
4         int n = nums.length;
5         for (int num : nums) {
6             count[num]++;
7         }
8         int runningSum = 0;
9         for (int i = 0; i <= 100; i++) {
10             int currentCount = count[i];
11             count[i] = runningSum;
12             runningSum += currentCount;
13         }
14         int[] result = new int[n];
15         for (int i = 0; i < n; i++) {
16             result[i] = count[nums[i]];
17         }
```

Saved

Ln 17, Col 10

☒ Testcase | ☒ Test Result**Accepted** Runtime: 0 ms☒ Case 1☒ Case 2☒ Case 3

Input

`nums =``[8,1,2,2,3]`

Output

`[4,0,1,1,3]`

3099. Harshad Number

Solved

Easy Topics Companies Hint

An integer divisible by the **sum** of its digits is said to be a **Harshad** number. You are given an integer x . Return *the sum of the digits* of x if x is a **Harshad** number, otherwise, return -1 .

Example 1:

Input: $x = 18$

Output: 9

Explanation:

The sum of digits of x is 9. 18 is divisible by 9. So 18 is a Harshad number and the answer is 9.

Example 2:

Input: $x = 23$

Output: -1

Explanation:

The sum of digits of x is 5. 23 is not divisible by 5. So 23 is not a Harshad number and the answer is -1.

Constraints:

• $1 \leq x \leq 100$

227 25 ?

6 Online

Code

Java Auto

```
1 class Solution {
2     public int sumOfTheDigitsOfHarshadNumber(int x) {
3         int sum=0;
4         for (int n=x;n>0;n/=10)
5             sum +=n%10;
6         return (x %sum==0)? sum:-1;
7     }
8 }
```

Saved

Ln 1, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

$x =$

18

Output

9

Expected

9

2114. Maximum Number of Words Found in Sentences

Easy

Topics

Companies

Hint

A **sentence** is a list of **words** that are separated by a single space with no leading or trailing spaces.

You are given an array of strings `sentences`, where each `sentences[i]` represents a single **sentence**.

Return *the **maximum number of words** that appear in a single sentence*.

Example 1:

Input: `sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]`

Output: 6

Explanation:

- The first sentence, "alice and bob love leetcode", has 5 words in total.
- The second sentence, "i think so too", has 4 words in total.
- The third sentence, "this is great thanks very much", has 6 words in total.

Thus, the maximum number of words in a single sentence comes from the third sentence, which has 6 words.

Example 2:

Input: `sentences = ["please wait", "continue to fight", "continue to win"]`

Output: 3

Explanation: It is possible that multiple sentences contain the same number of words.

In this example, the second and third sentences (underlined) have the same number of words.

</> Code

Java ▾ Auto

```
1 class Solution {
2     public int mostWordsFound(String[] s) {
3         int maxWords = 0;
4         for(String str:s){
5             String arr[]=str.split(" ");
6             int len=arr.length;
7             maxWords=Math.max(maxWords,len);
8         }
9         return maxWords;
10    }
11 }
12 }
```

Saved

Ln 3, Col 26

☒ Testcase | >_ Test Result

You must run your code first

1979. Find Greatest Common Divisor of Array

Easy

Topics

Companies

Hint

Given an integer array `nums`, return the ***greatest common divisor*** of the *smallest number and largest number in* `nums`.

The ***greatest common divisor*** of two numbers is the largest positive integer that evenly divides both numbers.

Example 1:

Input: `nums = [2,5,6,9,10]`

Output: 2

Explanation:

The smallest number in `nums` is 2.

The largest number in `nums` is 10.

The greatest common divisor of 2 and 10 is 2.

Example 2:

Input: `nums = [7,5,6,8,3]`

Output: 1

Explanation:

The smallest number in `nums` is 3.

The largest number in `nums` is 8.

The greatest common divisor of 3 and 8 is 1.

Example 3:

Input: `nums = [3,3]`

Output: 3

Explanation:

Code

Java | Auto

```
1 class Solution {
2     public int findGCD(int[] nums) {
3         int min = Integer.MAX_VALUE;
4         int max = Integer.MIN_VALUE;
5         for (int num : nums) {
6             if (num < min) {
7                 min = num;
8             }
9             if (num > max) {
10                max = num;
11            }
12        }
13        return gcd(min, max);
14    }
15    private int gcd(int a, int b) {
16        while (b != 0) {
17            int temp = b;
```

Saved

Ln 12, Col 10

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =
[2,5,6,9,10]

Output

2

2535. Difference Between Element Sum and Digit Sum of an Array

Easy

Topics

Companies

Hint

You are given a positive integer array `nums`.

- The **element sum** is the sum of all the elements in `nums`.
- The **digit sum** is the sum of all the digits (not necessarily distinct) that appear in `nums`.

Return the **absolute difference** between the **element sum** and **digit sum** of `nums`.

Note that the absolute difference between two integers `x` and `y` is defined as `|x - y|`.

Example 1:

Input: `nums = [1,15,6,3]`

Output: `9`

Explanation:
The element sum of `nums` is `1 + 15 + 6 + 3 = 25`.
The digit sum of `nums` is `1 + 1 + 5 + 6 + 3 = 16`.
The absolute difference between the element sum and digit sum is `|25 - 16| = 9`.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `0`

Explanation:
The element sum of `nums` is `1 + 2 + 3 + 4 = 10`.
The digit sum of `nums` is `1 + 2 + 3 + 4 = 10`.
The absolute difference between the element sum and digit sum is `|10 - 10|`.

Java

Auto

```
1 class Solution {
2     public int differenceOfSum(int[] nums) {
3         int elementSum = 0;
4         int digitSum = 0;
5         |
6         for (int num : nums) {
7             elementSum += num;
8             while (num > 0) {
9                 digitSum += num % 10;
10                num /= 10;
11            }
12        }
13        return Math.abs(elementSum - digitSum);
14    }
15 }
```

Saved

Ln 5, Col 8

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

[1,15,6,3]

Output

9